



Jisr (جسر)

Full Project Overview — Technical Report

Project	An AI writing companion that helps a young person draft a first message to a real person
Event	Ai4LY National Codathon 2026, Libya Artificial Intelligence Forum
Team	Mohamed Thabet (Team Leader), Rayan, Muatz, Shima
Date	8 October 2026

Generated from `REPORT.md` by `docs/scripts/build_overview_pdf.py`, so this document and the report say the same thing. Every figure in it comes from a command listed in Section 14 or in `safety/evidence.md`.

Contents

1. Executive Summary	9. Applicability in Libya
2. Problem Alignment	10. Limitations and Future Work
3. Product Principles and Boundaries	11. Attribution and Citations
4. System Architecture	12. Team and Ownership
5. Components and Team Contributions	13. Development Log
6. What the AI Actually Does	14. Reproducing the Prototype
7. Safety and Privacy Design	15. Conclusion
8. Evaluation and Results	

1. Executive Summary

Many young people in Libya who are under stress never tell anyone. The obstacle is rarely a lack of people to turn to. It is the first sentence: finding words that ask for help without alarming the listener or exposing too much.

Jisr targets exactly that moment. The user picks one or more everyday stress topics (exams, family, work, relationships, sleep, money, other), picks who they want to reach (a friend, a sibling, a parent, a trusted adult, a counsellor), and may add a few optional words. Jisr then proposes **three editable Arabic drafts in three tones, Gentle, Direct and Formal**. The user edits, copies or shares the message through their own messaging app. Nothing is ever sent automatically, and every flow ends by pointing the user toward a real person, not further into the app.

Jisr is built around four commitments:

- **Safe before helpful.** Every input is screened for crisis language *before* any drafting, on the phone and again on the server. If risk is detected, drafting stops and a support card is shown.
- **Never clinical.** Drafts are checked after generation, and any diagnostic, medical or treatment language is blocked.
- **Private by design.** The backend is stateless, personal identifiers are scrubbed before text reaches the model, and the model key never leaves the server.
- **Works when the network does not.** If the AI service is unavailable, the user still gets three safe, ready-made drafts.

The prototype is a four-part system delivered by four team members working in parallel: a mobile-first Arabic (RTL) front end, a stateless FastAPI AI backend deployed on Render, a safety layer with dialect-aware crisis detection and an output filter, and a trust and documentation layer that lets users and evaluators see what the AI did.

2. Problem Alignment

2.1 The challenge

The Codathon asks for an AI solution that helps young people in Libya manage psychological stress, access reliable guidance, and reach appropriate support early and safely. The solution must give AI a meaningful role, protect privacy, be practical in Libya, and must not diagnose conditions or replace qualified professionals.

2.2 The specific problem we chose

The team chose **one problem and one core experience**: *the gap between feeling stressed and telling someone*.

Three frictions stack up at that point:

1. **Stigma.** Distress is often kept inside the family, and putting a name on it can feel socially risky.
2. **Overwhelm.** When stress is high, structuring a clear message is hard.
3. **Articulation cost.** A message that invites support without causing panic takes emotional effort that the person may not have.

Specialist care is limited and unevenly distributed, and it should not be the first response to everyday academic or family stress. The most immediate support is usually already in the young person's circle. Jisr helps them reach it.

2.3 How Jisr maps to the brief

Brief requirement	How Jisr responds
Manage everyday and academic stress	Topic chips cover exams, family, work, relationships, sleep, money and other
Recognise when to seek support	The same-session prompt offers help writing a message once a topic is chosen
Connect to trustworthy support	Persistent "Talk to Someone Now" route and a verified-contacts-only support card
Protect privacy	Stateless backend, identifier scrubbing, on-device handling, nothing sent automatically
No diagnosis, no replacement of professionals	Output blocklist of clinical terms; every flow hands off to a human
Meaningful AI	Three concrete AI functions that templates cannot perform (Section 6)
Practical in Libya	Arabic RTL, dialect-aware safety, offline fallback, shares through apps people already use

3. Product Principles and Boundaries

Principle	What it means in the product
Human-to-human handoff	The AI is never the destination. The final screen encourages continuing with a real person in a messaging app.
Author agency	The AI proposes. The user edits and decides. Nothing is sent without the user's own action.
Safe before helpful	Risk screening runs before drafting. When risk is found, drafting is suspended.
Zero diagnostic ambition	No labels, scores, condition names or treatment guidance appear in outputs.
Private by default	No accounts, no stored user text on the server, no secrets in the repository.
Honest boundaries	The capture screen states what the product is not (a doctor, a therapist, a diagnosis tool, a replacement for specialists or an emergency service) and its advice for users under 18. The support card shows only contacts the team has verified.

Explicitly out of scope: medical diagnosis, medication advice, treatment plans, replacing therapists, doctors or emergency services, collecting unnecessary sensitive data, and any use of AI without a clear function.

4. System Architecture

4.1 Overview



A key design decision: the **front end, the safety layer and the Arabic content read the same shared files directly** (`src/i18n/ar.json`, `safety/support-card.json`, `safety/plain-templates.json`). There are no per-component copies, so the interface, the backend fallbacks and the safety content cannot drift apart.

4.2 Request flow

1. The user selects topics and a recipient, and optionally writes a few words.
2. **Identifier scrubbing.** Phone numbers (seven or more digits, Western or Arabic-Indic), email addresses, @handles and names (after phrases such as «اسمي», or from a list of common names) are replaced with placeholders. Family words such as «بابا» are kept, because the message needs them. The app scrubs before sending and the server scrubs again, using the same rules from one shared file (`safety/identifiers.json`). What was removed is returned so it can be reviewed.
3. **Risk check.** The phrase list runs first. The model then reads the same text to judge cases the list does not contain, including indirect wording.
 - If *either* the list or the model finds risk, the response reports risk and **no drafts are written**.
 - If the model does not answer in time, a phrase match still stands, and an **unanswered check is treated as a risk, not ignored**.

- The drafting route runs the same check again on the server before any drafting, so text is never drafted unchecked, even if the app skipped the first call. If the model cannot answer there, the server returns templates instead of drafting.
 - After the support card, the user may still continue to a note. They get the ready-made templates, and the flagged text is not sent to the model.
4. **Drafting.** The model produces three Arabic versions in Gentle, Direct and Formal tones for the chosen recipient.
 5. **Output check.** Each draft is scanned for clinical wording, condition names and treatment language. Failing drafts are blocked.
 6. **Fallback.** If the model cannot be reached, the server returns three ready-made templates for the chosen recipient instead of an empty error.
 7. The user edits the draft, then copies it or shares it. Nothing is sent automatically.
 8. **Faithfulness check.** After the user picks a draft, the server compares it with their original words and returns only the phrases that appear in both. If nothing can be shown or the model does not answer, the list is empty, and the three drafts remain available.

4.3 Service contract

The backend exposes three routes with a fixed request and response shape, agreed before implementation so the front end and backend could be built in parallel:

Route	Purpose
POST /api/check-risk	Phrase check plus model check; returns whether risk is present
POST /api/generate-drafts	Scrub, risk check (no drafts if risk), draft in three tones, output check, template fallback
POST /api/faithfulness	Returns phrases shared between the chosen draft and the user's own words

The live service runs at <https://jisr-api.onrender.com> (interactive docs at [/docs](#)).

5. Components and Team Contributions

5.1 Front end — Rayan (Person 1)

Rayan built the complete user journey in **Next.js and React**, designed for phones first, with full **Arabic right-to-left support** and the team's agreed Jisr colors and branding. The front end runs on its own using fallback data, and then connects to the live backend.

Area	What was built
Intro screen	Jisr mobile intro image, smooth transition into the app, a start button, and automatic advance after a few seconds. "Talk to Someone Now" is available even here.
Topic selection	Topic chips from the shared Arabic file, multi-select, with clear visual change on selection.
Optional writing	A free-text box that the user can skip entirely and continue with chips alone.

Area	What was built
Recipient selection	Recipient options from <code>ar.json</code> . The choice also selects the correct fallback message later.
Same-session trigger	A prompt offering help writing a message, filled with the selected topic. If the user chooses "not now", the prompt does not reappear for the same topic selection in that session.
Human support route	A persistent "Talk to Someone Now" button that stays on every step and opens the support card <i>separately</i> from the drafting flow.
Safety card	Reads <code>safety/support-card.json</code> . Only contacts marked verified are shown. With no verified contacts, the fallback safety message is shown. No contact is hardcoded in the front end.
Drafts	Reads <code>safety/plain-templates.json</code> and offers Gentle, Direct and Formal tones. Fallback text changes with recipient and topic. The user can edit before doing anything.
Copy and share	Copy to clipboard, plus the mobile Web Share option (<code>navigator.share()</code>), which falls back to copying where sharing is unsupported. Nothing is sent automatically.
Final handoff	A closing screen that encourages continuing with a real person in a messaging app.

Main files: `src/app/page.js`, `src/app/globals.css`, `src/app/layout.js`, `public/brand/jisr-intro-mobile.png`, plus the front-end project setup files.

Integration plan: after merging, the team re-tests the full flow, safety cases, real AI drafts, API-failure fallback, and mobile and RTL rendering.

5.2 AI backend — Muatz (Person 2)

Muatz designed the end-to-end request path and built the backend as a **FastAPI service that calls the Gemini model**, with the model key kept on the server and the user's text never stored between requests.

- **Identifier scrubbing before drafting.** Phone numbers, email addresses, @handles and names are replaced (family words are kept), and the removed items are returned for review.
- **Two-stage risk check.** The phrase list runs first. The model then judges cases the list does not contain, including indirect wording. Either one can raise risk.
- **Fail-safe behavior.** A timeout never silently passes a message. A phrase match stands, and an unanswered check is treated as a risk.
- **Post-generation output check.** Drafts containing clinical wording, condition names or treatment language are blocked.
- **Graceful degradation.** If the model is unreachable, three ready-made templates for the chosen recipient are returned instead of an error.
- **Faithfulness check.** After the user selects a draft, the server returns only phrases that appear in both the draft and the user's original words, with an empty list as the safe default.
- **Deployment.** The backend is merged on `main` and deployed on Render, configured from the `backend` folder, so the whole team calls the same three routes at one address. The model key lives on the host and is not in the repository.

End-to-end measurement. The risk check was run on the development set (`dev-set.json`): **11 of 11 crisis examples were caught**, and 6 non-crisis examples were also flagged. Section 8 explains why that set is an upper bound and gives the held-out results.

5.3 Safety layer — Shima (Person 3)

Shima owns the Guardian Layer, the set of files and checks that decide when Jisr must stop drafting and what it must never say.

Crisis phrase list (`crisis-phrases.json`). - Grew from 23 to **60 phrases**. - Covers both `روحي` and `نفسي` forms (for example `نأذي نفسي` and `نأذي روحي`). This closed a real gap: the earlier list missed `نبي نأذي روحي` because it only contained `نفسي`. - Adds short key words that catch many sentences, such as `انتحار` and `إنهاء حياتي`. - Adds Latin-letter spellings (`nabgi nmot` , `nentahar`) and English forms (`kill myself` , `end my life`), which also catch mixed Arabic and English. - Adds a **benign_idioms** list of harmless expressions such as `نموت من الضحك`. These are removed before matching, so a friendly "نبي نموت من الضحك" no longer triggers a false alarm. The model-based risk check still reads the full original text.

Forbidden clinical terms (`forbidden-terms.json`). - Grew from 28 to 47 terms in the safety update, with additions such as `طبيب نفسي` , `مرض نفسي` , `تشخيص` , `مكتتب` and `نوبة هلع` , and stands at **55 terms** in the final lexicon. - Everyday words such as `قلقان` and `ضغط` are **deliberately not banned**, so natural messages like "أنا قلقان من الامتحانات" still work.

Fallback templates (`plain-templates.json`). - 15 ready-made messages used whenever the AI is unavailable. - Wording is **gender-neutral** for boys and girls (`نحس بضغط` , `نحتاج`) and free of slashes, so messages are ready to send. - A `topic_labels` list fills `[topic]` with natural phrases (for example `أمور العائلة`) instead of raw button text. - All **105 variants (15 templates × 7 topics)** were tested: none contain medical words and none trigger the crisis check.

Support card (`support-card.json`). - Fields aligned to the agreed contract (`title_ar` , `message_ar` , `contacts` , `fallbackMessage_ar`) so the front end can read them directly. - Only **confirmed** contacts appear in `contacts` . The Red Crescent entry was moved to `pending_contacts` because it is not yet confirmed. - The fallback message is short and clear: speak with someone you trust now, and go to the nearest emergency department if in immediate danger.

Shared tooling.

File	Role
<code>lib/normalize.mjs</code>	Cleans text before every comparison so that all spellings match the same way
<code>lib/crisis-check.mjs</code>	Removes benign idioms, then looks for any crisis phrase; returns yes or no and the matching phrase
<code>lib/output-check.mjs</code>	Checks each AI draft for forbidden terms; returns pass or fail and which words were found
<code>evaluate.mjs</code>	Measures crisis recall and false alarms, and reports ambiguous sentences separately
<code>selftest.mjs</code>	Automated checks that all files load, no phrase is duplicated, spelling variants are caught, idioms are not, medical words are blocked, templates are safe, and no unconfirmed phone number can appear

The **normalizer** is what makes Arabic matching reliable. It removes diacritics (نبي → نبي) and stretched letters (نبي → نبي), unifies alef forms (ا → آ إ إ), maps ي → ى and ه → ه , lowercases English, removes apostrophes, and turns punctuation and emoji into spaces. The crisis list and the medical-word list both use it, and the backend uses the same rules, so every part of the system matches text in the same way.

Evaluation discipline. Shima renamed `test-set.json` to `dev-set.json` once the phrase list had been improved using those exact sentences, because they could no longer fairly test it. A fresh set is required for headline numbers (Section 8).

5.4 Trust, localization and documentation — Mohamed Thabet (Team Leader, Person 4)

As Team Leader, Mohamed defined the shared contracts and conventions, and delivered the trust views, the Arabic content and the competition documentation.

- **Three trust views** that make the AI's behavior inspectable by users and judges:
- `BaselineComparison.tsx` lets the viewer switch between the AI-adapted draft and a static generic template, which shows what the AI adds.
- `FaithfulnessView.tsx` highlights which parts of a draft come from the user's own words, so the user can check what the draft took from them before editing it.
- `OutboundPreview.tsx` shows exactly what would leave the device after identifier scrubbing.
- **Libyan Arabic localization** (`src/i18n/ar.json`): the master dictionary with culturally grounded phrasing for all 7 stress topics, 5 recipient options, 3 tones, disclaimers and interface prompts, reused directly by every other component.
- **Client-side identifier scrubbing** for phone numbers (including +218 , 091 , 092 , in ASCII and Arabic-Indic digits), emails, @handles and names, kept identical to the server through a shared rules file and a shared test fixture.
- **Competition documentation:** this report, the README submission portal, the 5-minute pitch script (`submission/PITCH_DECK.md`), the committee Q&A playbook (`submission/COMMITTEE_QA.md`), and `CITATIONS.md` .
- **Release and verification:** the consolidated `submission/` directory, the composite test command, and the pre-compiled Android build for sideloading.

6. What the AI Actually Does

A recurring risk in this domain is *decorative AI*. Jisr uses AI for four jobs that forms and templates cannot do well, and keeps deterministic code in charge of everything safety-critical.

#	AI function	Why rules or templates are not enough
1	Structuring messy free text. Turns chaotic input (for example ما نقدرش نكمل مع الامتحانات وبيا يضغط علينا) into a coherent message that separates the situation, its impact and the implicit need.	Forms need tidy inputs that an overwhelmed person will not provide.
2	Tone and audience adaptation. Adjusts vocabulary, honorifics and social distance for a friend, a parent or a counsellor, in three tones.	String substitution sounds robotic and misses social nuance.

#	AI function	Why rules or templates are not enough
3	Faithfulness alignment. Returns the phrases that a chosen draft shares with the user's own words, so the user can see which parts came from them.	A template generator cannot verify where its words came from.
4	Indirect-risk detection. The model reads the full text for distress that the phrase list does not contain, including euphemism and indirect wording.	Keyword matching alone misses new slang and metaphor.

Division of labor. The model handles language: tone, structure and indirect meaning. **Deterministic code handles safety:** the crisis phrase list, the clinical blocklist, the support card and the template fallback. The model never decides whether to show a phone number, and it never gets the last word on what a draft may contain.

Why we did not fine-tune a model

We chose a hosted foundation model with careful prompting plus deterministic guardrails over custom training, for three reasons:

1. **Safety control.** A deterministic filter before and after generation gives guarantees that a fine-tuned model cannot. Fine-tuning on a small dataset can also weaken general language ability and safety behavior.
2. **Data ethics.** Training on young people's distress messages would require collecting exactly the sensitive data this project avoids. All of our evaluation material is synthetic.
3. **Practicality.** There was no large local dataset available, and running a local model on typical phones would be heavy on download size and battery.

7. Safety and Privacy Design

7.1 Defense in depth

Stage	Mechanism	Failure behavior
Before drafting	Phrase list (Arabic, Libyan dialect, Latin letters, English) plus model risk check, on the phone and again inside the drafting route	A phrase match always stands. In the risk check, a missing model answer counts as risk; inside the drafting route it produces templates, so text is never drafted unchecked.
Risk found	Drafting stops. Support card appears.	Only verified contacts are shown. Otherwise a fixed fallback message appears. The user may continue to a note made from templates; the flagged text is not sent to the model.
After drafting	Clinical-term blocklist on every draft	A failing set of drafts is regenerated once, then replaced by safe templates.
Model unavailable	Ready-made templates for the chosen recipient	The user always receives three usable drafts.

Stage	Mechanism	Failure behavior
Always	"Talk to Someone Now" on every step	Opens the support card independently of the drafting flow.

Designed to over-trigger. The risk check is tuned to err toward showing the support card. A false alarm costs a harmless card. A missed crisis could cost far more.

7.2 Honest support information

The team set a strict rule for contacts: **a phone number is shown only if a named team member has personally verified it.** In Libya, unverified or disconnected crisis lines are a real hazard, because a person in distress who reaches a dead line may feel more abandoned.

Accordingly, `contacts` is deliberately empty in the shipped support card. The Red Crescent entry sits in `pending_contacts` until confirmed, and the card shows an honest fallback that directs the user to someone they trust and to the nearest emergency department:

«لا نستطيع عرض رقم لم نتحقق منه. تحدث مع شخص تثق فيه الآن، وإذا كنت في خطر مباشر توجه لأقرب قسم طوارئ»
 ("We cannot show a number we have not verified. Talk to someone you trust right now, and if you are in immediate danger, go to the nearest emergency department.")

The self-test also checks that no unconfirmed phone number can appear in any output.

7.3 Privacy measures

- **Stateless backend.** No database, no accounts, no disk logging of user text, no storage between requests.
- **Secrets stay on the host.** The model key is held by the hosting service and is not in the repository.
- **Identifier scrubbing.** Phone numbers, emails, @handles and names are replaced on the phone before any text leaves it, and again on the server. Family words are kept. The scrub result is returned for review.
- **Transparency.** The Outbound Preview shows what leaves the device.
- **Nothing sent automatically.** Copy and share are user actions, handed to the user's own apps.
- **Minimal data.** Topic chips make the app usable with no typing. Free text is optional.
- **Private record, off by default.** The optional history on the phone holds chip topics and times only, never text or recipients. It expires after 1, 7 or 30 days, is excluded from Android backup, and is erased when switched off.
- **Logs and keys.** The server log records only path, status and timing; the web server's own access log, which would record IP addresses, is switched off. The model key is sent in a request header, never in a URL.
- **Model provider.** The identifier-free text is processed by Google's Gemini API. On the free tier Google may use it to improve its products, so a deployment for real users needs a paid-tier key (see `CITATIONS.md`).
- **Young users.** No account is needed. The app advises users under 18 to use Jisr with a trusted adult's knowledge.

7.4 Data provenance

No chat logs, social media posts or patient records were collected or used. All crisis phrases, idioms and development sentences were written by team members. The held-out sentences (`safety/heldout-set.json`) were written by the AI code reviewer (Claude Code) and are labelled as such in the file. Third-party models and libraries are listed in Section 11.

8. Evaluation and Results

8.1 What we measure

- **Crisis recall:** of the messages that signal danger, how many are caught.
- **False-alarm rate:** of ordinary stress messages, how many are wrongly flagged.
- **Output filter reliability:** whether any forbidden clinical term can pass through.
- **Template safety:** whether fallback messages are themselves safe.

8.2 Results

Check	Result	How to read it
Combined risk check on the held-out set (<code>heldout-set.json</code> : 26 crisis, 30 ordinary; not used for tuning)	26 of 26 crisis sentences caught (95% CI 87–100%); 3 of 30 ordinary sentences flagged (10%, CI 3.5–26%)	The model alone caught all 26 with no false alarm; the phrase list alone caught 6 of 26. See Section 8.4.
Combined risk check on the development set (<code>dev-set.json</code>)	Backend owner's run: 11 of 11 caught, 6 of 14 non-crisis flagged. Re-run on 8 October against the live service: 11 of 11 caught, 1 of 14 flagged	An upper bound, because the phrase list was tuned on these sentences. 95% Wilson lower bound on recall for 11/11 is about 74%. The model is not deterministic, which is why the false-alarm count moved.
Fallback templates	105 filled variants (15 × 7 topics): none contain medical words or trigger the crisis check	Fallback drafts are safe by construction.
Safety self-test (<code>selftest.mjs</code>)	Began at 394 automated checks in the safety update; the final suite reports 450 passing, 0 failing	Covers file loading, duplicate phrases, spelling variants, idioms, blocklist, templates and unconfirmed numbers.
Output filter	Every forbidden term in the blocklist is intercepted in testing	Deterministic, so a listed term cannot pass.
Cross-implementation parity	The on-device crisis checker matches the Node.js engine across all phrases, variants and idioms	Different parts of the system agree on what counts as a crisis.

8.3 The development set, and why we do not headline it

The 25-sentence practice set (`dev-set.json`) is where the phrase list was improved. On it, the original list caught 6 of 10 crisis messages and the improved list catches 10 of 10 with no false alarms.

We do not report that as performance, because the list was tuned using those same sentences. It shows that the fixes work on known gaps, such as the missing `روحي` variants, but it cannot estimate accuracy on unseen text. This is the reason the team separated development material from evaluation material.

8.4 Held-out results

On 8 October 2026 the risk check was measured on a held-out set of 64 synthetic sentences (`safety/heldout-set.json`): 26 crisis, 30 ordinary and 8 ambiguous, in Libyan dialect, standard Arabic, Latin letters, English and mixed Arabic and English. The set was not used to tune the phrase list or the prompt, and was not edited after the first run. It was written by the AI code reviewer, who had read the phrase list, so it is held-out but **not blind**.

Layer	Crisis caught (recall)	Ordinary messages flagged
Phrase list alone	6 of 26 (23%, 95% CI 11–42%)	3 of 30 (10%, CI 3.5–26%)
Model alone (live service, <code>gemini-flash-lite-latest</code>)	26 of 26 (100%, CI 87–100%)	0 of 30 (0%, CI 0–11%)
Combined, as shipped	26 of 26 (100%, CI 87–100%)	3 of 30 (10%, CI 3.5–26%)

Two findings matter. First, the phrase list does not generalise far beyond the sentences it was tuned on, so the model does most of the detection on new wording: plans, farewell behaviour, plain English and danger from others. This is the clearest evidence for the AI's role in safety (Section 6, function 4). Second, the phrase list still earns its place as the instant, offline first pass and as the floor when the model is down, but it causes all of the combined false alarms.

Because `/api/check-risk` fails closed, a model outage would look like a run of detections. A benign canary was checked before, halfway through and after each run, and all came back clean. Commands and per-item results are in `safety/evidence.md` and `safety/results/` .

8.5 Interpreting the flagged non-crisis examples

In the backend owner's development-set run the combined check flagged 6 of 14 non-crisis examples. In the 8 October runs it flagged 1 of 14 on the development set and 3 of 30 on the held-out set, and all 3 held-out false alarms came from the phrase list: «انتحار» in a sentence about a TV scene or a research article, and «نختفي» about leaving social media. We treat this as the cost of a deliberately cautious design: a flagged message shows the support card and does not harm the user. The phrase list itself was built to avoid avoidable false alarms through its benign-idiom exemption and its decision not to ban everyday words like `فلقان` and `ضغط` . Reducing these false alarms without lowering recall, for example by adding such contexts to the benign-idiom list after native review, is the first item in our future work.

8.6 Usability targets

Jisr is built to shorten the path from "I need to say something" to "I have a message ready." We set the following as targets for field testing with real users, and we **do not claim them as measured results**:

- A first draft on screen within roughly a minute of opening the app.
- A majority of users who pick a topic go on to copy or share a message.
- Users editing less than about 30% of a draft, as a sign the draft is a good starting point.

8.7 Evaluation gaps we are open about

- The held-out set was written by an AI that had seen the phrase list, so it is not blind, and AI-written sentences may be easier for an AI classifier than real messages. A **blind set** written by people who have not seen the list is still needed.
- Each set was run once; the model is not deterministic, and `gemini-flash-lite-latest` can change without notice.
- The sample sizes so far are small, so confidence intervals are wide.
- We have not yet run a study with real young users or with Libyan mental health professionals.

9. Applicability in Libya

Libyan reality	Jisr's response
Dialect and script mixing. Young people write in Libyan dialect, standard Arabic, Latin letters and English, often in the same sentence.	Crisis phrases cover Arabic, dialect, Arabizi and English variants. A shared normalizer handles diacritics, stretched letters and alef forms.
Gendered language. Direct translations often assume a speaker's gender.	Fallback templates use gender-neutral phrasing.
Stigma around "mental health" labels.	The product never uses clinical labels. Messages are about ordinary stress topics such as exams, family or work.
Unreliable connectivity and power.	The client falls back to local templates when the service is unreachable or does not answer within a minute (the free hosting can take that long to wake), so the user still gets three safe drafts.
Messaging apps are the real support channel.	Users share through the system share sheet into the apps they already use, and nothing is sent automatically.
Scarce verified hotlines.	The app does not invent contacts. It shows verified contacts only and otherwise points to a trusted person or the nearest emergency department.
Family and trusted elders matter.	Recipient options include parents, siblings and "a trusted adult", alongside friends and counsellors.

10. Limitations and Future Work

Limitations

- **Dialect coverage.** Strong regional slang or very local expressions may be missed by both the phrase list and the model.
- **False alarms.** The cautious risk check flags some non-crisis messages (Section 8.5).
- **Offline detection.** The phrase list alone caught 6 of 26 held-out crisis sentences. When the phone is offline only the phrase list runs, which is why the "Talk to Someone Now" button never depends on either check.
- **Identifier scrubbing.** Misspelled names can evade pattern-based scrubbing. The Outbound Preview helps the user catch these.
- **Faithfulness check.** It is deliberately conservative: it returns only phrases shared with the user's words, and returns an empty list when unsure.
- **Evaluation depth.** The held-out set is synthetic and not blind; a blind set and real-user testing are still outstanding.
- **Support contacts.** No external hotline has been verified yet, so the card relies on the honest fallback until partners are confirmed.
- **Model dependence.** Draft quality depends on a hosted model, and the service depends on connectivity (mitigated by templates). When the model is down, the risk check fails closed: every message gets the support card, with the option to continue to templates.

Future work

1. Build a blind test set written by people who have not seen the phrase list, with native and clinical review; reduce phrase-list false alarms without losing recall; and grow the on-device list so offline detection depends less on the model.
2. Partner with Libyan psychosocial organizations to verify real contacts and move entries out of `pending_contacts`.
3. Review phrases, tones and templates with Libyan counsellors and young people.
4. Run a small usability study against the targets in Section 8.6.
5. Add recipient types such as academic advisors and workplace supervisors.
6. Explore on-device generation so that drafting works fully offline.
7. Provide an aggregated, privacy-preserving view that could help institutions understand community needs without exposing individuals.

11. Attribution and Citations

All third-party models, services and libraries used in the prototype are open-source or accessed through public APIs, and are listed below. Full formal citations are in `CITATIONS.md`.

Component	Used for	Notes
Google Gemini API (<code>gemini-flash-lite-latest</code>)	Risk judgment, three-tone drafting, faithfulness check	Accessed through the server. The key is held on the host and not in the repository.

Component	Used for	Notes
FastAPI and Uvicorn	Backend web service	Python, stateless service with typed request and response schemas
Render	Hosting of the backend	Service configured from the <code>backend</code> folder
Next.js and React	Mobile-first web front end with Arabic RTL support	Front-end client
Expo and React Native	Native mobile packaging and native share sheet	Used for the sideloadable Android build
Web Share API (<code>navigator.share()</code>)	Sharing a draft to messaging apps	Falls back to clipboard copy
Node.js	Safety tooling, normalizer, evaluation harness and self-tests	Shared by the safety layer

Content provenance. The crisis phrase list, benign-idiom list, forbidden-term list, fallback templates, topic labels, Arabic dictionary and the development sentences were written by the team. The held-out sentences were written by the AI code reviewer. No external datasets, scraped text or patient data were used.

Development tools. Claude Code (Anthropic) was used on 8 October 2026 for a repository review, the fixes listed in `docs/REVIEW_REPORT.md` , and the held-out evaluation sentences (see `CITATIONS.md`).

12. Team and Ownership

Member	Role	Delivered
Mohamed Thabet	Team Leader, Person 4: trust views, Arabic localization and documentation	Shared contracts and conventions, <code>BaselineComparison.tsx</code> , <code>FaithfulnessView.tsx</code> , <code>OutboundPreview.tsx</code> , <code>ar.json</code> , client-side scrubbing, this report, README, pitch script, committee Q&A, <code>CITATIONS.md</code> , submission packaging
Rayan	Person 1: front end	Mobile-first Next.js and React interface with Arabic RTL, intro, topic chips, optional text, recipient selection, same-session prompt, persistent human route, support card, three-tone drafts, copy and share, human handoff
Muatz	Person 2: AI core and backend	FastAPI service, Gemini integration, identifier scrubbing, two-stage fail-safe risk check, post-generation output check, template fallback, faithfulness check, Render deployment
Shima	Person 3: safety, Guardian and evidence	60-phrase crisis list with idiom exemption, clinical blocklist, gender-neutral fallback templates, support card with verified-only contacts, shared normalizer, crisis and output checks, evaluation harness and self-test suite

13. Development Log

The team worked in parallel on four workstreams, with a fixed interface agreed up front and short stages that were each reviewed before the next began.

Stage	Backend (Muatz)	Safety (Shima)	Front end (Rayan)	Trust and docs (Mohamed)
1. Contracts	Planned the request path and fixed the three-route contract	Agreed field names and the <code>[topic]</code> placeholder format	Agreed shared files as the single source of content	Set repository conventions and shared setup
2. Foundations	FastAPI setup; identifier scrubbing; phrase-based risk check	First crisis, forbidden-term and template files	Mobile-first RTL shell and branding	Arabic localization dictionary
3. Parallel build	Gemini-based risk check alongside the phrase list; output check; template fallback	23 → 60 crisis phrases; 28 → 47 forbidden terms; idiom exemption; gender-neutral templates; normalizer	Intro, chips, optional text, recipients, same-session prompt, human route	Trust views
4. Integration	Faithfulness check; deployment to Render on <code>main</code>	Evaluation harness, self-test; support-card contract and pending-contacts rule	Safety card, drafts in three tones, copy and share, handoff screen; PR to <code>main</code>	Documentation, submission packaging
5. Verification	Ran the risk check on the development set: 11 of 11 caught	Dev-set separated from evaluation; fresh set required	Post-merge testing of safety cases, API failure and RTL	Report, pitch and Q&A finalized

Key decisions

1. **Hybrid over fine-tuning.** Use a hosted model for language and deterministic code for safety.
2. **Fail-safe defaults.** A missing risk answer counts as risk. A missing model answer produces templates, not an error.
3. **Verified contacts only.** An empty contact list with an honest fallback is safer than an unverified number.
4. **One source of truth.** The front end, backend fallbacks and safety content read the same shared files.
5. **One normalizer everywhere.** Every text match in the system uses the same normalization rules.
6. **Honest evaluation.** Tuned development sentences are never presented as headline accuracy.

Review and hardening (8 October)

An independent review of the repository (Claude Code, at the team's request) led to these changes, listed in `docs/REVIEW_REPORT.md`: the risk check repeated inside the drafting route, one identifier rule set shared by the app and the server, the private record off by default with expiry, continuing to templates after the support card, the stated limits and the minors notice in the app, the held-out evaluation in Section 8.4, and backend hardening (key in a header, no IP access log, a model-call cap and a text-length limit).

14. Reproducing the Prototype

Detailed setup, requirements and usage are in the root `README.md`. In brief:

```
# Safety layer checks
node safety/selftest.mjs
node safety/evaluate.mjs safety/dev-set.json
node safety/evaluate.mjs safety/heldout-set.json

# All app, privacy and safety suites; backend tests
npm test
npm run test:backend

# Front end (Next.js)
npm install
npm run dev      # http://localhost:3000

# Backend (FastAPI)
cd backend
pip install -r requirements.txt
uvicorn app.main:app --port 8000
```

The model key is supplied through an environment variable on the host and is not in the repository. The hosted backend is available at <https://jisr-api.onrender.com>, with interactive documentation at [/docs](#).

15. Conclusion

Jisr shows that AI in mental health is most useful as a **bridge, not an authority**. It uses a language model for what models do well, finding the right words in the right tone, and relies on deterministic, inspectable code for what must never go wrong: recognizing danger, avoiding clinical claims, and being honest about what support exists. It does this while collecting no more data than it needs, and it works when the network does not. By helping a young person write the first sentence, Jisr aims to return care to where it belongs, in human hands.